

# OKF vs. RAG + VectorDB: A Hands-On Comparison

Two architectures for making an undocumented dataset agent-consumable,  
built and measured side by side

google-okf-tutorial

2026-07-05

## Contents

|                                                                     |          |
|---------------------------------------------------------------------|----------|
| <b>Executive Summary</b>                                            | <b>1</b> |
| <b>1. Background — the problem both approaches solve</b>            | <b>2</b> |
| <b>2. Architecture A: OKF</b>                                       | <b>2</b> |
| 2.1 What OKF is . . . . .                                           | 2        |
| 2.2 What was built (google_okf_zero_to_mastery.ipynb) . . . . .     | 3        |
| <b>3. Architecture B: Flat RAG + VectorDB (+ Agentic Retrieval)</b> | <b>3</b> |
| 3.1 What was built (agentic_rag_chromadb.ipynb) . . . . .           | 3        |
| 3.2 A related, retired experiment . . . . .                         | 3        |
| <b>4. Side-by-Side Comparison</b>                                   | <b>4</b> |
| 4.1 Structural comparison . . . . .                                 | 4        |
| 4.2 What each notebook actually measured . . . . .                  | 5        |
| 4.3 The pattern that shows up twice . . . . .                       | 5        |
| <b>5. Decision Guidance</b>                                         | <b>6</b> |
| <b>6. Limitations of This Comparison</b>                            | <b>6</b> |
| <b>7. References</b>                                                | <b>7</b> |

## Executive Summary

This report compares two working ways to make an undocumented dataset queryable by an AI agent, both built end-to-end on the same real dataset with the same local models in the google-okf-tutorial repository:

1. **OKF** (Google’s Open Knowledge Format v0.1) — a structured, spec-conformant knowledge bundle, consumed by a dense-retrieval RAG agent (google\_okf\_zero\_to\_mastery.i

2. **Flat RAG + VectorDB** — no knowledge-structuring layer at all, just text chunks in ChromaDB, consumed by both a simple retrieval pipeline and an autonomous, tool-calling agentic pipeline (`agentic_rag_chromadb.ipynb`).

**Headline finding:** at the scale tested (16 knowledge items, 8-14 evaluation questions), *simple* retrieval already recovers 88-92% of correct answers regardless of which knowledge-representation approach it sits on top of. The sophistication in both “smarter” variants tested — an advanced multi-technique RAG pipeline and an agentic tool-calling loop — earns its cost almost exclusively on **multi-hop questions**, and is neutral-to-harmful everywhere else. Neither architecture “wins” outright; they answer different engineering questions.

---

## 1. Background — the problem both approaches solve

Real datasets accumulate undocumented structure: table meanings, join keys, business metrics, and data-quality quirks that live only in the heads of whoever last worked with them. An AI agent asked a question about that data has no single place to look, and inherits whatever inconsistent, partial documentation exists — or none.

Two fundamentally different engineering answers to “how do we fix that” are tested here:

- **Structure the knowledge first, retrieve second.** Curate a durable, versioned, spec-conformant representation of what’s known about the data — then build retrieval on top of *that*.
- **Skip structuring, retrieve directly.** Dump whatever content exists into a vector database and let retrieval (optionally augmented with agentic autonomy) do all the work at query time.

Both were built against the same dataset — the [Olist Brazilian E-Commerce Public Dataset](#) (real, anonymized, 9 relational tables, CC BY-NC-SA 4.0) — with the same local models (qwen3.5:4b for generation, nomic-embed-text for embeddings, both  $\leq 4B$  parameters, no cloud APIs), so the comparison below is about architecture, not data or model quality.

---

## 2. Architecture A: OKF

### 2.1 What OKF is

The [Open Knowledge Format v0.1](#) represents knowledge as a directory of Markdown files with YAML frontmatter. The entire mechanism is: type is the one required frontmatter field; everything else — title, description, resource, tags, timestamp — is recommended but optional. Concepts link to each other via ordinary Markdown links. Two reserved filenames, `index.md` and `log.md`, provide progressive disclosure and a change history. Conformance requires exactly two things: every concept parses as

valid frontmatter, and type is non-empty — everything else (broken links, unknown types, missing optional fields) must be tolerated by a compliant consumer.

## 2.2 What was built (`google_okf_zero_to_mastery.ipynb`)

1. A from-scratch OKF library: document model, frontmatter render/parse, bundle I/O, `index.md/ log.md` generation, link extraction, and a real conformance validator against SPEC.md §9.
2. Deterministic foreign-key discovery (column-name matching with a `zip_code_prefix` special case) and six real business metrics computed directly with pandas.
3. A local LLM enrichment agent that drafts each concept’s prose — **facts come from code, the model only writes narrative**, and every generated metric claim is checked against the real computed number before being accepted (with a retry-then-deterministic-fallback path).
4. A local RAG discovery agent: dense embedding retrieval (FAISS) + a listwise LLM rerank call with a guaranteed floor of raw retrieval hits.

**Result:** 16 conformant OKF concepts (1 dataset, 9 tables, 6 metrics), 0 conformance errors, 0 warnings.

---

## 3. Architecture B: Flat RAG + VectorDB (+ Agentic Retrieval)

### 3.1 What was built (`agentic_rag_chromadb.ipynb`)

The same enrichment discipline (facts from pandas, prose from the LLM, every claim checked) but the *output* is a plain `{id, text, metadata}` dict per item — no frontmatter, no bundle hierarchy, no parseable cross-links. Sixteen such documents are ingested into a [ChromaDB](#) collection via a custom Ollama embedding function, then consumed two ways:

- **Simple RAG** — one retrieval call, one generation call. Architecturally identical in shape to notebook 1’s baseline, just over a flat corpus instead of an OKF bundle.
- **Agentic RAG** — native Ollama tool-calling (the interleaved reasoning/acting pattern from [Yao et al. 2022](#), [ReAct](#), in the broader family surveyed by [Singh et al. 2025](#)). The model gets one tool, `search_knowledge_base`, and decides for itself whether one search is enough, bounded at 4 iterations.

### 3.2 A related, retired experiment

An earlier notebook (since removed from this repository at the user’s request, but its findings carried forward into this comparison and into `agentic_rag_chromadb.ipynb`’s design) built a *non-agentic* but materially more sophisticated RAG stack on top of the **OKF** bundle: RAG-Fusion multi-query expansion, HyDE, hybrid dense+BM25 retrieval fused with Reciprocal Rank Fusion, OKF-graph-link expansion, and pointwise

(not listwise) LLM reranking. Its numbers are referenced in §4.3 as a second, independent data point on the same underlying question — does “smarter” retrieval actually pay for itself — because the pattern it found turned out to reproduce almost exactly in this repo’s current agentic experiment.

## 4. Side-by-Side Comparison

### 4.1 Structural comparison

|                               | OKF                                                                                                        | Flat RAG + VectorDB                                                                                                    |
|-------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| What it is                    | A knowledge <i>representation</i> spec — markdown + YAML frontmatter, bundles, concepts, cross-links       | An <i>inference-time retrieval technique</i> over unstructured chunks in a vector database                             |
| Layer                         | Authoring / storage                                                                                        | Query-time consumption                                                                                                 |
| Storage                       | Plain files, git-diffable, human-browsable on GitHub with zero tooling                                     | A running (even if embedded/in-process) vector database process                                                        |
| Structure                     | Explicit: typed frontmatter, # Schema/# Joins/# Citations sections, a real link graph                      | None inherent — whatever metadata fields the pipeline author remembered to attach                                      |
| Consumption without retrieval | Possible — a small bundle fits wholesale in an LLM’s context, browsable via <code>index.md</code>          | Not really — there’s no directory to browse; the vector index <i>is</i> the access path                                |
| Conformance / validation      | A real, checkable spec (SPEC.md §9); this project’s validator actually runs and asserts                    | No equivalent concept — “is this corpus well-formed” isn’t a question a vector DB asks                                 |
| Graph-aware retrieval         | Possible — an OKF bundle’s real links can be walked (the retired advanced-RAG notebook did this)           | Not natively — there is nothing to walk; “related tables” only exist as prose the retriever might or might not surface |
| Setup cost                    | A YAML/markdown parser, nothing else                                                                       | A vector DB dependency (ChromaDB — embedded SQLite + ONNX runtime)                                                     |
| Where it shines               | Long-lived, versioned, human- <i>and</i> -agent-curated knowledge that outlives any one retrieval pipeline | Fast to stand up, works over content nobody structured on purpose (PDFs, wikis, scraped pages)                         |

## 4.2 What each notebook actually measured

| Notebook                     | Corpus            | Consumption                                                        | n (eval)                     | Key metric                                                          |
|------------------------------|-------------------|--------------------------------------------------------------------|------------------------------|---------------------------------------------------------------------|
| 1 — OKF                      | 16 OKF concepts   | Dense retrieval + guaranteed-floor listwise rerank                 | 8 questions                  | Hit@1 88%, Hit@3/5 100%, groundedness 3/3                           |
| 2 — Flat + simple RAG        | 16 flat documents | Single retrieve + generate                                         | 12 answerable / 2 distractor | Retrieval hit 91.7%, citation-hit 83.3%                             |
| 2 — Flat + agentic RAG       | 16 flat documents | Tool-calling loop, $\leq 4$ iterations                             | 12 answerable / 2 distractor | Retrieval hit 91.7%, citation-hit 75.0%, 1.17 searches/q            |
| (retired) OKF + advanced RAG | 16 OKF concepts   | RAG-Fusion + HyDE + hybrid + RRF + graph-expand + pointwise rerank | 12 answerable / 2 distractor | Final-context hit 83.3% (vs. 91.7% baseline), MRR 0.822 (vs. 0.792) |

**These rows are not one controlled experiment** — different eval-set difficulty, different executions, different corpora. They are three independent measurements of a related question, and are presented as such, not averaged into a false single number.

## 4.3 The pattern that shows up twice

Two independently-built “smarter than naive” pipelines — one OKF-based (advanced RAG, retired), one flat-corpus-based (agentic RAG, current) — were each benchmarked against their own naive baseline on the *same* 14-question hard eval set. Both times, the added sophistication showed the identical shape of result:

| Category   | OKF: baseline → advanced (final-context hit) | Flat: simple → agentic (citation hit) |
|------------|----------------------------------------------|---------------------------------------|
| Direct     | 100% → 75%                                   | 100% → 75%                            |
| Paraphrase | 100% → 100%                                  | 100% → 67%                            |
| Multi-hop  | 66.7% → 100%                                 | 66.7% → 100%                          |
| Vague      | 100% → 50%                                   | 50% → 50%                             |

Multi-hop is the one category where *both* independently-designed “smarter” pipelines improved, by the identical margin (66.7% → 100%). Both also degraded on direct questions by roughly the same margin (100% → 75%). This is not a coincidence worth

ignoring: added retrieval machinery (whichever kind) is buying real value specifically where a single vector lookup structurally cannot find everything needed — and buying noise everywhere a single lookup was already sufficient.

---

## 5. Decision Guidance

**Reach for OKF (or a similar structured format) when:** - The knowledge is worth curating once and reusing for years, across multiple consuming systems. - Humans need to review, edit, and version the same artifacts an agent reads. - You want a checkable notion of “is this documentation complete/well-formed.” - The corpus is small enough that “just load it into context” is a real, competitive alternative to retrieval — OKF bundles are readable wholesale in a way flat vector stores are not.

**Reach for flat RAG + a vector DB when:** - The content already exists, unstructured, and structuring it first is not worth the time (PDFs, wikis, scraped pages, one-off analyses). - You need retrieval working today, over a corpus that will change shape often. - No one is going to hand-curate or review the corpus as a first-class artifact.

**Add agentic (tool-calling) retrieval when:** - Multi-hop questions are common in your actual query distribution — this is the one place both experiments in this repository found it reliably earns its cost. - You can afford 1.6-4.3× the LLM/embedding calls per question (measured here, not estimated) and have a way to cap iterations (both experiments bounded it, and never hit the cap). - You are prepared for it to occasionally *underperform* simple retrieval on easy questions, since a small ( $\leq 4B$ ) model doesn’t reliably know when its own extra effort is unnecessary.

**Skip agentic retrieval when:** - Most real questions are direct or vague — both experiments found extra machinery is neutral or harmful there, at real extra cost.

---

## 6. Limitations of This Comparison

- **Small corpus (n=16) and small eval sets (n=8 or n=14).** Percentage differences of one or two questions swing several points. None of these numbers should be read as precise population estimates.
- **Single model family.** Every result here uses qwen3.5:4b for generation and nomic-embed-text for embeddings. A larger or differently-trained model could plausibly change which category benefits from which technique — these findings describe *this* model scale, not models in general.
- **Not a controlled three-way experiment.** The OKF-advanced-RAG numbers came from a notebook that no longer exists in this repository (removed per a later request); they are quoted from that notebook’s own recorded run, not re-executed alongside this report.
- **Heuristic evaluation components.** Refusal-correctness and citation-hit are regex/substring checks, not judge-model or human evaluation — the same class of limitation disclosed inline in both live notebooks.

---

## 7. References

1. Open Knowledge Format v0.1 specification — [SPEC.md](#), [GoogleCloudPlatform/knowledge-catalog](#)
2. Google Cloud Blog — “[How the Open Knowledge Format can improve data sharing](#)”
3. Cormack, Clarke & Büttcher (2009). “[Reciprocal Rank Fusion outperforms Condorcet and Individual Rank Learning Methods.](#)” SIGIR '09.
4. Rackauckas (2024). “[RAG-Fusion: a New Take on Retrieval-Augmented Generation.](#)”
5. Gao et al. (2022). “[Precise Zero-Shot Dense Retrieval without Relevance Labels](#)” (HyDE), ACL 2023.
6. Edge et al., Microsoft Research. “[GraphRAG: Unlocking LLM discovery on narrative private data.](#)”
7. Sun et al. (2023). “[Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents](#)” (RankGPT).
8. Yao et al. (2022). “[ReAct: Synergizing Reasoning and Acting in Language Models.](#)”
9. Singh et al. (2025). “[Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG.](#)”
10. Dataset — [Brazilian E-Commerce Public Dataset by Olist](#) (Kaggle, CC BY-NC-SA 4.0).
11. `google_okf_zero_to_mastery.ipynb` and `agentic_rag_chromadb.ipynb` — the two executed notebooks in this repository this report is drawn from.